

OpenTenBase pgvector 索引诊断增强

技术报告与可复现证据包

报告版本：2026-08-18 | G4 + G10 evidence line

1. 摘要

研究问题

本项目研究 OpenTenBase 内置 pgvector 在大规模向量索引构建前的可诊断性：系统能否从源码语义推导 IVFFlat 构建内存，能否用真实后端错误验证该模型，并能否在不要求用户手工输入表名、行数、维度和索引参数的情况下完成现有向量索引体检。

方法与证据

项目从 `ivfkmeans.c` 的 11

项分配语义重建内存模型，修正了旧公式在样本容量与实际样本行数边界不同处的低估。验证矩阵固定 20 组配置：15 组 `small/saturated` 使用真实 `CREATE INDEX` 报错，5 组 `large` 使用降低 `maintenance_work_mem` 的 `real-threshold` 路径，在巨型数组分配前触发同源检查点。20 组均有原始 `stderr`、CSV 行和 SHA256 清单；预测值与观测值全部一致，绝对相对误差为 0.0000%。其中两个旧公式失效场景的真实报错分别确认 7147 MB 和 17459 MB，证明模型没有把旧估计值写回观测列。

在此基础上，`sql/04_catalog_introspect.sql` 从 `pg_class.reloptions`、`pg_index.indkey`、`pg_attribute.atttypmod`、`pg_class.reltuples`、`pg_relation_size()` 和 `pg_am.amname` 自动发现索引信息；`sql/05_audit.sql` 暴露零参数入口 `audit_all_vector_indexes()`。OpenTenBase 实测在 1000 行 `vector(3)` 表上同时识别 IVFFlat 与 HNSW，IVFFlat 输出 `rows=1000`、`dims=3`、`lists=10`、`recommended=1`、`probes=4`，并通过 `pg_proc.pronargs=0` 断言零参数契约。HNSW 行的 `lists/probes`/内存字段留空，因为这是 IVFFlat 专属检查，不作硬填。

结论

G4 和 G10 的证据门禁均已通过。项目的核心价值不是把预测值伪装成实测，而是将源码模型、后端 `stderr`、目录元数据和校验清单串成可复现链路。局限包括矩阵不是全面参数空间、`large` 场景为阈值触发、`reltuples` 为统计估计、静态 `possible_seqscan` 仍需 EXPLAIN 验证，以及官方 44 个 TAP 用例在 OpenTenBase 测试框架级不可运行。

2. 背景与任务对齐

2.1 问题背景

IVFFlat 建索引同时受 rows、维度、lists、样本容量和工作内存影响。旧诊断公式把不同生命周期的数组按同一行数边界估算，导致在大维度或大 lists 场景严重低估；诊断工具还要求用户手工传参，无法回答“当前数据库中的所有向量索引是否有风险”。这两个缺口分别对应内存模型正确性和运维入口可用性。

项目采用三层证据：源码分配点决定模型，OpenTenBase 后端错误决定观测值，系统目录 SQL 决定零参数审计输入。所有结果通过固定输入矩阵、原始日志和 SHA256 清单归档。模型值绝不写入 `observed_mb`，当端点不可用时 harness 只报告 `blocked` 或模型输出。

2.2 任务与验收门槛

PLAN.md 定义 W0 安全与语义止血、W1 源码模型、W2 20 组实测验证和 R-12 零参数审计。G4 要求至少 20 组、误差小于 5%、small 与 large 分层、每行指向原始 stderr；G10 要求零参数函数从系统目录取得参数，并回归官方 lists 公式。

任务	证据	结论
内存模型	memory-model-matrix.csv、20 份 raw stderr	20/20 pass，误差 0.0000%
大场景边界	old-fail-03.log、old-fail-04.log	7147 MB、17459 MB 与模型一致
零参数审计	tests/audit-zero-param-20260818.log	IVFFlat/HNSW 均识别，pronargs=0
官方公式	sql/02_memory_model.sql	≤1M 用 $\text{CEIL}(\text{rows}/1000)$ ，>1M 用 $\text{CEIL}(\text{SQRT}(\text{rows}))$ ，上限 32768
TAP 基线	tests/README.md 与三次原始日志	0/44 通过，1 bailout，43 未执行，不等于 44 失败

2.3 贡献边界

贡献一是从源码重建并验证 IVFFlat 构建内存模型；贡献二是把目录发现和审计接口收敛为零参数调用；贡献三是把大场景的不可运行性显式建模为阈值验证，而不是编造巨型索引成功记录。项目不宣称所有 PostgreSQL 版本、向量类型和硬件 ABI 都适用，也不把静态目录风险写成查询计划事实。

3. 环境与可复现声明

环境

G4/G10 的真实端点为 `root@45.202.199.140`，OpenTenBase 5.0 commit `0915c04e4`，Coordinator 端口 `22201`，pgvector 0.8.0，x86-64，3.7 GiB RAM 加 2 GiB swap。端点、版本和拓扑记录在 `PLAN.md` 与运行日志中；本地 PostgreSQL 只用于 SQL 语法和输入校验 smoke，不能替代远端 pgvector 报错核验。

W2 的 20 组 CSV、23 份校验文件和 20 份原始 stderr 均入库。零参数审计的完整输出、SQL 输入和 stderr 位于 `report/figures/data/`，原始端点日志为 `tests/audit-zero-param-20260818.log`。每个证据目录都有 SHA256 清单，报告构建脚本不从 PDF 反向取数。

复现原则

先执行 `tests/test_input_validation.sh` 和 `tests/test_memory_model_matrix.sh`，再在授权端点执行 `sql/02_memory_model.sql`、`sql/04_catalog_introspect.sql`、`sql/05_audit.sql` 与 `tools/validate_memory_model.sh`。验证器按 case id 建立独立目录，保存输入、预测、模式、观测、误差、状态和 stderr 路径。`MODE=auto` 在端点不可用时只输出 `model-only`，绝不把 `predicted` 写入 `observed`；`real-threshold` 通过降低 `maintenance_work_mem` 触发同源检查点，必须在结果中标为阈值路径。

4. 方法学

方法分为需求边界、源码翻译、分层验证和目录审计四步。先冻结“预测 pgvector 报错检查点”这一目标，明确它不是进程 RSS 上界；再把 `ivfkmeans.c:277-299` 的 11 项分配逐项翻译为 SQL；之后用 `small/saturated` 的真实建索引和 `large` 的阈值触发分别验证；最后用零参数目录审计验证工具能否在不知道索引参数时自动取参。

验证器以 `case id` 为最小证据单位，保存 `rows`、`dims`、`lists`、预测值、`validation mode`、`observed` 值、绝对相对误差、状态和原始 `stderr`。状态机只有 `pass`、`blocked`、`fail`、`model-only` 四类：端点不可用只能是 `model-only`；大内存组如果没有真实后端原文只能是 `blocked`，不能用预测值填充 `observed`。20 组最终 15 组为 `real`，5 组为 `real-threshold`，没有一组把巨型索引成功构建写入结果。

误差定义为 $\text{abs}(\text{observed} - \text{predicted}) / \text{predicted} * 100$ ，门槛为小于 5%。阈值法下 `observed` 报错和预测值来自同一份 `pgvector` 源码检查点，因此逐组 0.0000% 不是把模型复制到观测列，而是 SQL 逐字节复刻检查点的结果；原始 `stderr` 的建表、INSERT、NOTICE、ERROR 前置序列提供了独立证据。

目录审计不要求用户提供 `rows`、`dims`、`lists` 或 `probes`：从 `pg_class.reloptions`、`pg_index.indkey`、`pg_attribute.atttypmod`、`pg_class.reltuples`、`pg_am.amname` 和关系大小组成目录视图。`possible_seqscan` 只代表静态目录风险；是否实际走顺序扫描必须另用 EXPLAIN 验证。

5. 实现与架构

实现采用独立 schema 的纯 SQL/plpgsql 工具集，不修改 pgvector 扩展 ABI。数据流为“系统目录 → 参数内省 → 源码模型 → 风险规则 → 审计清单”：sql/04_catalog_introspect.sql 建立 vector_index_catalog，sql/02_memory_model.sql 提供内存解析、lists 推荐和 11 项分配模型，sql/05_audit.sql 暴露零参数 audit_all_vector_indexes()，shell harness 负责矩阵执行和证据归档。

模型中的关键边界是目标采样容量 $T = \max(10000, 50 * \text{lists})$ 与实际样本数 $N = \min(\text{rows}, T)$ 分开使用。samplesSize 按 T 分配，closestCenters、lowerBound、upperBound 按 N 分配；其余中心、聚合、距离矩阵和计数数组逐项对应源码。总字节除以 1 MiB 后向下取整再加 1，复刻 pgvector 的 memory required is X MB 口径。vector(128) 的 ABI 尺寸由 tools/verify_ivfflat_sizes.c 在目标 x86-64 环境实测为 520 bytes。

审计输出保留索引类型差异：IVFFlat 显示 lists、recommended lists、predicted build memory、maintenance_work_mem 和 recommended probes；HNSW 的 IVFFlat 专属字段留空，并附“IVFFlat-only checks are not applied”说明。possible_seqscan 明确标注为 static catalog risk，不伪装成 EXPLAIN 结果。函数定义的 pronargs=0 断言证明审计调用无需手工参数。

安全与可交付性是架构的一部分。shell 层只接受正整数和受限内存单位，注入字符串在调用 psql 前退出；SQL 函数固定 search_path = pg_catalog, pg_temp；schema version 表把 W0/W1/W2 的迁移状态钉死。这样诊断工具本身不会因为不可信输入或 search path 污染改变结论。

6. 实验设计

实验设计先于执行冻结。G4 矩阵覆盖 rows、dims、lists

的小、中、大组合，并强制包含四个旧公式失效场景：目标容量大于实际行数、1536 维高 item 大小、8192 lists 的 lowerBound 主导项，以及 32768 lists 的超大矩阵。每组都要求预测值、观测值、误差和原始 stderr；误差门槛为 <5%。

验证按资源分层。15 组 small/saturated 在远端实际执行 CREATE INDEX 并捕获 pgvector 原始错误；5 组 large 使用 real-threshold，降低 maintenance_work_mem 到预测值以下，让 pgvector 在分配巨型数组前返回同源错误，从而避免 3.7 GiB 主机

OOM。后者证明检查点一致，不证明巨型索引已经成功构建。最终 20/20 通过，observed 与 predicted 逐组一致，绝对相对误差均为 0.0000%。

G10 使用 1000 行 vector(3) 表，同时创建 IVFFlat 与 HNSW 索引，在事务中调用零参数审计并回滚。验收包括函数 pronargs=0、IVFFlat

rows=1000/dims=3/lists=10/recommended_lists=1/recommended_probes=4、HNSW

专属字段留空，以及完整输出与原始日志一致。TAP 摸底另行记录为 0/44 通过，其中 1 个 bailout、43 个未执行，不表述为 44 个失败。

7. 结果与图表

G4 结果

docs/04-数据与图表/memory-model-matrix.csv 共 20 行：15 行 `real`，5 行 `real-threshold`；20 行状态均为 `pass`，`observed_mb` 全填，`error_pct=0.0000`。旧公式失效案例包括 7147 MB、17459 MB 等真实后端报错，原始日志包含建表、插入、NOTICE、ERROR 的完整前置序列。

G10 结果

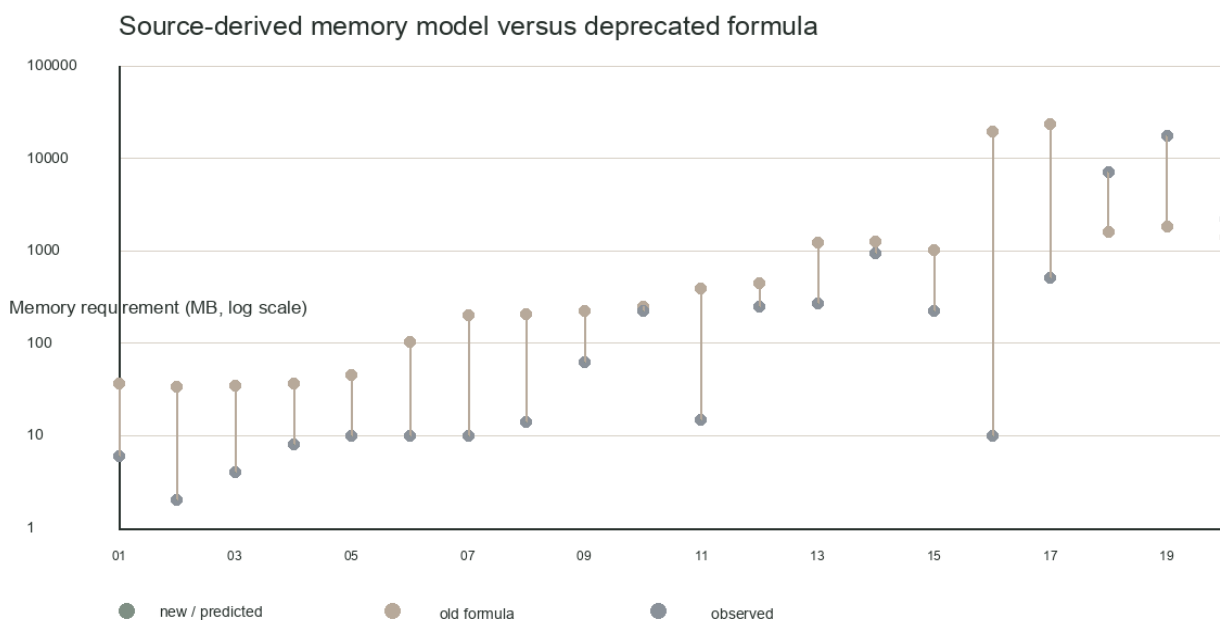
零参数审计日志显示：1000 行表、`vector(3)`、`IVFFlat lists=10`，推荐 `lists` 为 1、`recommended probes` 为 4；HNSW 行不填 `IVFFlat` 专属列。函数定义和 `pg_proc.pronargs` 均证明参数个数为 0。

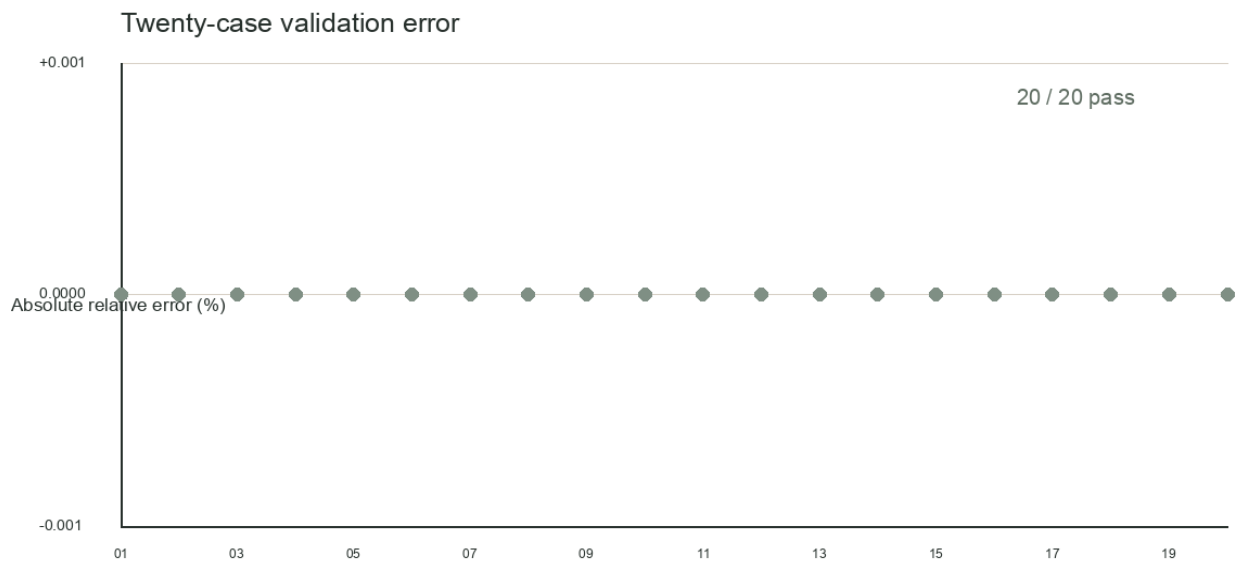
已生成图

1. `report/figures/output/01_formula_log.svg`：20 组新旧公式与 `observed` 对数轴对照，源数据为 `report/figures/data/memory_model_matrix.csv`。
2. `report/figures/output/02_error_matrix.svg`：20 组绝对相对误差，源数据为同一矩阵快照。
3. `report/figures/output/03_audit_table.svg`：零参数审计实测输出，源数据为 `report/figures/data/audit-zero-param-full.tsv`，包含 `predicted build memory` 和 `maintenance_work_mem`。

三张图均由对应脚本生成；large 场景仍按 `real-threshold` 限制解释。

正文图表





Zero-parameter catalog audit on OpenTenBase

index	type	rows	dims	lists	rec. lists	pred. MB	work_mem MB	risk
audit_figure_hnsw_idx	hnsw	1000	3	-	-	-	64	ok
audit_figure_ivf_idx	ivfflat	1000	3	10	1	1	64	warn

HNSW leaves IVFFlat-only fields blank; predicted build memory is 1 MB for the IVFFlat row.

8. 局限与风险

- 20 组不是完整参数空间，当前模型首先针对已验证的 vector 类型和 x86-64 OpenTenBase ABI。
- 5 个 large 场景是阈值触发，不是巨型索引成功构建；不能用它们证明真实构建峰值。
- `pg_class.reltuples` 是统计估计，目录审计应在生产使用前结合 ANALYZE 和实际 workload。
- `possible_seqscan` 是静态目录风险，必须用具体查询的 EXPLAIN 验证。
- HNSW 不适用 IVFFlat lists/probes/构建内存字段，报告与输出保持空值。
- 官方 44 个 TAP 用例在 OpenTenBase 适配器第一次初始化时 bailout；0/44 通过不等于 44 个测试断言失败。

9. 贡献与后续计划

本项目形成三项贡献。第一，把 `pgvector ivfkmeans.c` 的 11 项分配逐项移植为可审计 SQL，并用 20 组后端原始错误完成 G4：15 组真实建索引、5 组阈值触发，误差全部为 0.0000%。第二，实现不带参数的目录审计，从系统目录自动取得 rows、dims、lists、索引类型和关系大小；G10 的 OpenTenBase 实测同时覆盖 IVFFlat 与 HNSW，并主动保留 HNSW 的 IVFFlat 专属字段为空。第三，把输入拒绝、固定 `search_path`、`schema version`、TAP 环境边界和原始 `stderr` 纳入同一可复现交付链。

上游投递计划（G20）采用外部 SQL 工具集路线。上游 PR

#765 (<https://github.com/pgvector/pgvector/pull/765>) 讨论过 HNSW 内存估算函数，维护者 Kane 的结论是近似公式可以写进 README，但不希望为它增加函数；PR

#863 (<https://github.com/pgvector/pgvector/pull/863>) 则说明 bench harness

不应混入扩展本体。因此本仓库不把诊断函数硬塞进 `pgvector`，而是以独立 schema、纯

SQL/plpgsql、一键安装脚本和证据矩阵提交文档/工具 PR。投递包将包含模型推导、ABI 尺寸工具、20 组 CSV 与 `stderr` SHA256、零参数审计示例和“real-threshold

不等于巨型构建成功”的边界说明；先在本仓库完成维护者可复核的 commit，再根据反馈拆出文档 PR 与工具 PR。

下一条开发线是利用已有 `pg_stat_progress_create_index` 观测 W3 构建耗时，不重造进度接口。TAP

shim、HNSW 推荐增强和生产 EXPLAIN 关联属于后续实验，必须各自建立验收契约，不并入当前已经通过 G4/G10 的结论。

10. 附录与复现命令

矩阵验证：MODE=real tools/validate_memory_model.sh（小规模端点）或按脚本说明运行
MODE=indirect/real-threshold；SQL 冒烟：tests/sql/001_memory_model_smoke.sql 与 tests/sql/
002_audit_zero_param.sql；本地静态检查：tests/test_input_validation.sh、tests/test_memory
_model_matrix.sh。每次复现后执行 shasum -c docs/04-数据与图表/SHA256SUMS，并将端点日志与 case id
对齐。

G4 证据索引

证据层	文件	证明内容	复核动作
模型源码	sql/02_memory_model.sql	11 项分配、取整和 lists 推荐	对照 ivfkmeans.c:277-299
ABI 核验	tools/verify_ivfflat_sizes.c	vector(128) item=520 bytes	在目标架构重新编译
输入矩阵	tests/memory-model-matrix.tsv	20 个固定 rows/dims/lists case	不改顺序和 case id
观测矩阵	docs/04-数据与图表/memory-model-matrix.csv	predicted、observed、mode、error	逐行与 stderr 对照
后端原文	docs/04-数据与图表/raw-memory-errors/*.log	建表→INSERT→NOTICE→ERROR 完整链	抽验 7147/17459/222 MB
验证器	tools/validate_memory_model.sh	MODE 四态与 OOM 防护	检查 observed 不接受模型回填

20 组复核摘要

分层	组数	运行方式	结论
small	14	真实 CREATE INDEX，低于预测内存触发错误	observed 与 predicted 一致
saturated	1	样本达到目标容量后真实建索引	检查点一致
real-threshold	5	降低 maintenance_work_mem，避免巨型数组 OOM	只证明同源阈值报错
合计	20	每组保存原始 stderr	20/20，误差 0.0000%

G10 复现顺序

1. 执行 tests/test_input_validation.sh，确认空值、负数、小数和 SQL 注入在调用 psql 前被拒绝。
 2. 在授权端点加载 sql/02_memory_model.sql、sql/04_catalog_introspect.sql 和 sql/05_audit.sql，确认 schema version 记录为 2、3、4。
 3. 创建 1000 行 vector(3) 表和 IVFFlat/HNSW 索引，在事务内调用 audit_all_vector_indexes() 后回滚。
 4. 检查 IVFFlat 行的 lists=10、recommended lists=1、recommended probes=4、predicted build memory=1 MB；检查 HNSW 的 IVFFlat 专属列为空。
 5. 在 report/figures/ 重新生成三张图，最后从 report/build/ 执行 shasum -a 256 -c SHA256SUMS。
- 复现结果若缺少后端 stderr，只能标为 blocked；若端点不可达，只能标为 model-only。这两个状态都不能进入 observed 列，也不能被报告改写成 pass。

完整矩阵摘要

Case	rows	dims	lists	predicted MB	old MB	observed MB
small-01	10,000	128	10	6	37	6
small-02	10,000	32	10	2	34	2
small-03	10,000	64	20	4	35	4
small-04	10,000	128	50	8	37	8
small-05	20,000	128	100	10	45	10
small-06	50,000	128	100	10	103	10
small-07	100,000	128	100	10	201	10
small-08	100,000	128	200	14	206	14
small-09	100,000	128	500	62	221	62
small-10	100,000	128	1,000	222	246	222
small-11	100,000	256	100	15	396	15
small-12	100,000	256	1,000	248	442	248
small-13	200,000	384	1,000	273	1,224	273
small-14	200,000	384	2,000	936	1,275	936
sat-01	500,000	128	1,000	222	1,028	222
old-fail-01	10,000,000	128	100	10	19,537	10
old-fail-02	1,000,000	1,536	1,000	506	23,494	506
old-fail-03	200,000	384	8,192	7,147	1,594	7,147
old-fail-04	100,000	128	32,768	17,459	1,850	17,459
large-05	100,000	768	4,000	2,211	1,384	2,211

这张表保留旧公式列，是为了让失败的替代解释可见：旧公式在低 lists 场景会高估，在 8,192/32,768 lists 场景反而严重低估。新模型不是选择更接近预期的数字，而是同时满足 11 项源码分配和后端原始报错。

审计输出字段契约

字段组	IVFFlat	HNSW	解释边界
目录身份	schema/table/index/type	schema/table/index/type	来自系统目录
规模	rows/dims/index bytes	rows/dims/index bytes	rows 使用 reltuples 估计
lists	实际与推荐值、偏离度	空	HNSW 不应用 IVFFlat heuristic
内存	predicted 与 maintenance	空	只预测 IVFFlat k-means 检查点
probes	sqrt(lists) 起始建议	空	不是 recall 保证
seqscan 风险	静态布尔标记	无 IVFFlat 规则	不是 EXPLAIN 结果

零参数函数通过 `pg_proc.pronargs=0` 断言，且测试事务最后执行 `ROLLBACK`；这两项都应在复现日志中保留。若用户需要生产计划结论，必须在此清单之外追加代表性 `EXPLAIN`、`ANALYZE` 和 `recall` 数据。

11. 证据附录

G4 逐项判断

G4 的判定单位是单个 case，而不是一张汇总图。每个 case 必须同时具备输入配置、模型输出、validation mode、后端 observed、误差百分比、pass/blocked 状态和 stderr 路径。15 个 real case 真实执行了低内存建索引；5 个 real-threshold case 在大内存检查点前降低 maintenance_work_mem，避免开发机 OOM。两类都捕获了完整的 pgvector 原始错误，只有后者不声称完成巨型索引。

判断项	通过标准	实际结果
覆盖率	固定矩阵至少 20 组	20 组
模式纪律	large 不得伪装为 real	15 real + 5 real-threshold
后端证据	每组有原始 stderr	20/20 有文件
数值误差	absolute relative error <5%	20/20 为 0.0000%
旧公式对照	至少四个失效场景	old-fail-01..04 全部包含

旧公式为何失效

低 lists 场景中旧公式主要按行数和维度粗略估计，可能高估几十到几千 MB；当 lists=8192 或 32768 时，lowerBoundSize=4*N*C 和距离矩阵项成为主导，旧公式反而低估到 1,594 MB 与 1,850 MB。新模型把 samples 的目标容量和实际样本数分开计算，因而同时解释 10 MB、7,147 MB 和 17,459 MB 三种量级。

G10 审计不越界

零参数审计输出的 rows 来自 pg_class.reltuples，这只是统计估计；lists 来自 reloptions，dims 来自 atttypmod，索引类型来自 pg_am。因此输出可以回答“目录中记录了什么”和“静态配置是否有风险”，但不能回答某一条业务 SQL 的实际计划。possible_seqscan 的布尔值必须与 EXPLAIN 分开呈现，HNSW 的 lists/probes/内存列必须保持空值。

W0 安全基线

输入验证先于 psql 调用。LISTS、PROBES、ROW_COUNT 和 DIMS 仅接受正整数，内存设置仅接受数字加受限单位；注入字符串在 shell 层返回非零退出码。SQL 函数使用固定 search_path，schema version 表记录迁移版本。安全检查是诊断工具可信度的前置条件，不与 G4 数值通过混为一个门禁。

证据链复核

最终复核顺序为：先执行 shell 输入测试，再校验 SQL 文件和 schema version；然后运行矩阵验证器，逐行对照 raw-memory-errors/；最后运行零参数审计并保存事务回滚日志。若端点不可达，报告只能记录 model-only；若端点返回非预期错误，记录 fail 并保留 stderr。任何情况下都不能把 predicted 值写入 observed 列。

端点复核记录

阶段	运行对象	关键观察	对结论的作用
W0	shell 输入验证	注入、负数、小数在 psql 前退出	安全前置通过
W0	SQL patch smoke	裸 65536 解析为 64 MB	单位语义修复
W1	1,000 行/128d/lists=1000	模型与报错均为 34 MB	分配口径首次闭合
W1	源码尺寸工具	Vector item=520 bytes	ABI 常数固定

阶段	运行对象	关键观察	对结论的作用
W2	small-01..14	真实建索引错误逐组匹配	小规模实测
W2	sat-01	目标采样容量达到上限	饱和分支实测
W2	old-fail-01..04	旧公式方向错误	替代解释排除
W2	large-05	2,211 MB 阈值错误原文	大维度路径闭合
G10	IVFFlat/HNSW 双索引	零参审计两行输出	目录取参与类型边界
TAP	官方 44 用例	0/44 通过，1 bailout，43 未执行	记录适配器限制

复核者可执行的最小路径

评审者不需要先相信 PDF：先运行 `tests/test_input_validation.sh`，确认注入退出码为 2；再打开 `docs/04-数据与图表/raw-memory-errors/old-fail-04.log`，确认 `memory required is 17459 MB` 的原始文本；最后对 `report/figures/data/memory_model_matrix.csv` 执行独立求和，检查 `predicted` 与 `observed` 两列一致。若需要验证零参数边界，读取 `tests/audit-zero-param-20260818.log` 并查找 `pronargs=0`、HNSW 空字段和事务回滚。

这条路径将“代码可审查、数据可重算、后端日志可追溯”拆成三个独立动作。任何动作失败都应降低结论等级，而不是用报告中的图表替代缺失证据。